

Descripción Técnica - Autoridad de Tiempo y Hora

2023-07

Requisitos del sistema.....	2
Arquitectura del sistema.....	3
Diseño del sistema.....	4
Actores.....	4
Flujo general de solicitudes.....	5
Generador de sellos de tiempo.....	7
Firma.....	9
Autenticación del solicitante y control de acceso.....	11
API.....	11
Uso por Usuario:.....	11
Protección de la privacidad.....	12
Reloj.....	12
Registro de auditoría.....	13

Requisitos del sistema

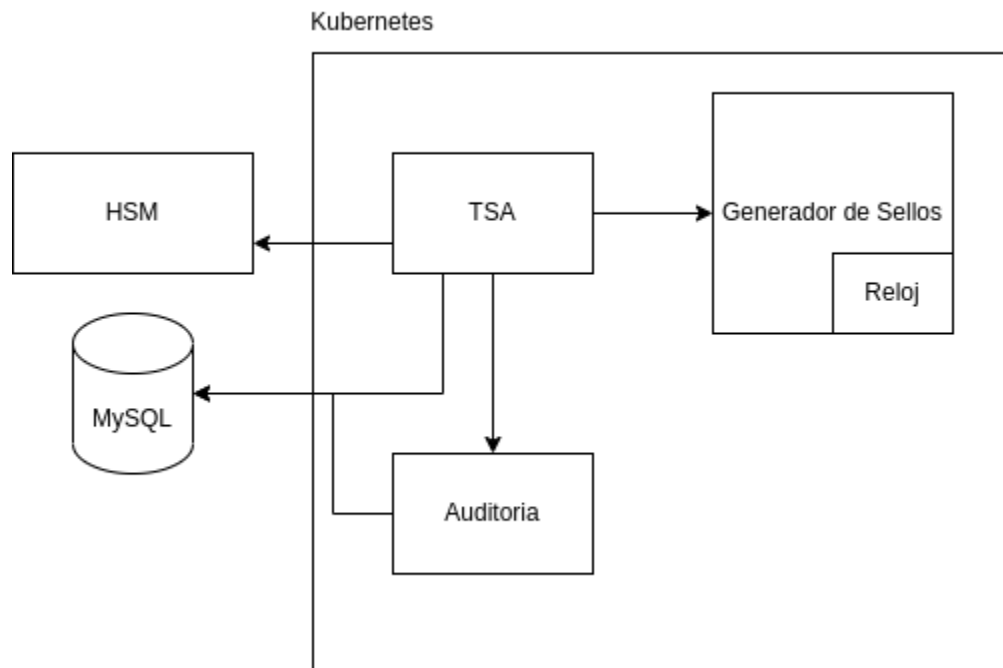
1. Generación de sellos de tiempo
2. Autenticación del solicitante
3. Protección de la privacidad
4. Firma digital
5. Control de acceso
6. Monitoreo y auditoría

Arquitectura del sistema

Se recomienda la siguiente Arquitectura para este servicio. El servicio de TSA, Auditoría y Generador de Sellos montados dentro de un servicio de Kubernetes, para permitir su fácil escalabilidad.

El Reloj será parte del servicio de Generador de Sellos, y utilizará Chrony mantener su hora sincronizada con Amazon Time Sync

La TSA podrá realizar solicitudes de firma al HSM y la base datos MySQL sería un servicio externo, se recomienda que tenga alta disponibilidad y los controles necesarios para asegurar la seguridad de la información

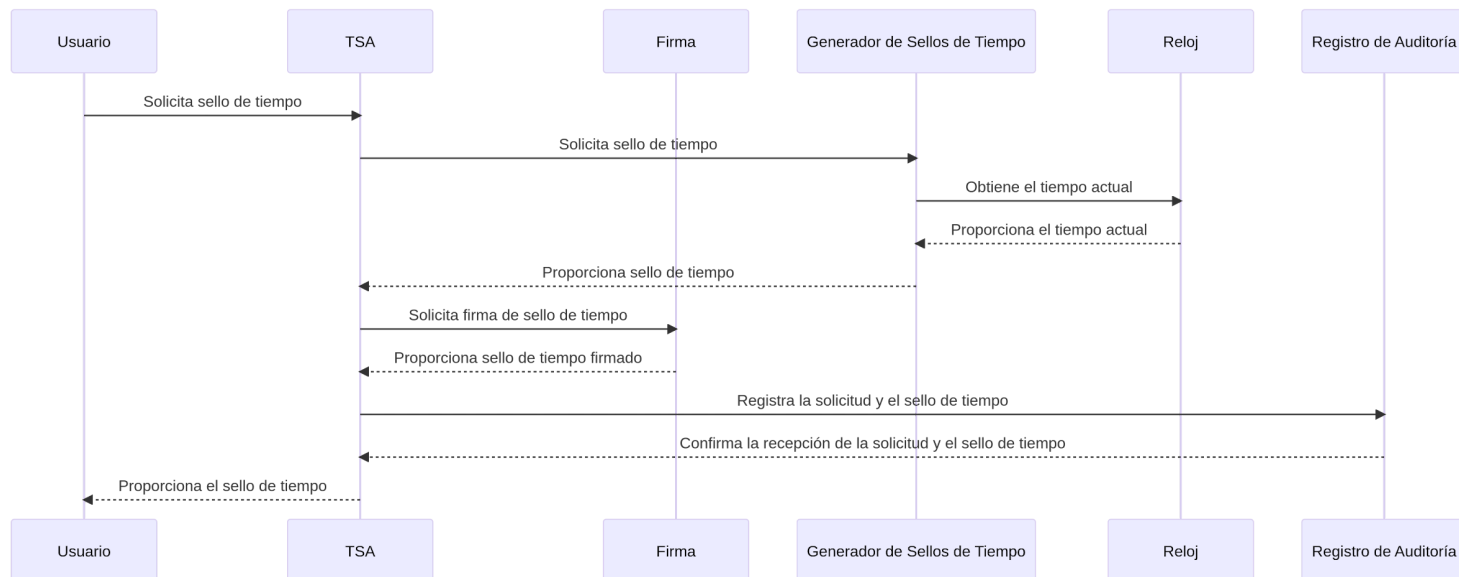


Diseño del sistema

Actores

1. TSA: Expone el servicio de sellado de tiempo. Su función principal es la de recibir solicitudes de sellado de tiempo, procesarlas y luego responder con un sello de tiempo. También maneja la autenticación de las solicitudes y la gestión de registros y auditorías.
2. Generador de Sellos de Tiempo: Toma la información de la solicitud y utiliza esta información para generar un sello de tiempo
3. Reloj: Fuente confiable de tiempo y hora actuales
4. Registro de Auditoría: Mantiene un registro de todas las solicitudes de sello de tiempo y respuestas que son solicitadas y recibidas por la TSA.
5. Usuario: Envía solicitudes de sello de tiempo a la TSA

Flujo general de solicitudes



Solicitud del Usuario: El proceso comienza cuando el usuario solicita un sello de tiempo a la Autoridad de Sello de Tiempo (TSA).

La solicitud se enviará por medio de un HTTP request con Content Type application/timestamp-query. Y un TimestampReq codificado en ASN1 como el body del request

Solicitud a la TSA: Tras recibir la solicitud del usuario, la TSA realiza una solicitud al Generador de Sellos de Tiempo.

Consulta al Reloj: El Generador de Sellos de Tiempo, para generar un sello único, solicita el tiempo actual al Reloj.

Respuesta del Reloj: El Reloj responde a la solicitud del Generador de Sellos de Tiempo proporcionándole el tiempo actual.

Creación del Sello de Tiempo: Usando el tiempo proporcionado por el Reloj, el Generador de Sellos de Tiempo crea un sello único y se lo proporciona a la TSA.

Registro de la Acción: La TSA registra la solicitud original del usuario y el sello de tiempo recién generado en el Registro de Auditoría.

Solicitud de firma: La TSA solicita la firma del sello generado

Confirmación del Registro de Auditoría: El Registro de Auditoría confirma a la TSA la correcta recepción y registro de la solicitud y del sello de tiempo.

Entrega del Sello de Tiempo:La TSA proporciona el sello de tiempo al usuario que lo solicitó originalmente

Generador de sellos de tiempo

El Generador de sellos de tiempo recibirá las solicitudes de sellos de tiempo. Estas solicitudes HTTP tendrán el siguiente formato:

```
Unset
POST HTTP
Content-Type: application/timestamp-query
Content-Length: [longitud del cuerpo de la solicitud]

[TimeStampReq en formato binario]
```

El TimeStampReq tendrá que cumplir con el formato definido por el RFC 3161 de la IETF

```
Unset
TimeStampReq ::= SEQUENCE {
    version                INTEGER { v1(1) },
    messageImprint          MessageImprint,
    -- a hash algorithm OID and the hash value of the data to be
    -- time-stamped
    reqPolicy              TSAPolicyId          OPTIONAL,
    nonce                  INTEGER              OPTIONAL,
    certReq                BOOLEAN              DEFAULT FALSE,
    extensions              [0] IMPLICIT Extensions OPTIONAL
}

MessageImprint ::= SEQUENCE {
    hashAlgorithm          AlgorithmIdentifier,
    hashedMessage          OCTET STRING
}

TSAPolicyId ::= OBJECT IDENTIFIER

https://www.ietf.org/rfc/rfc3161.txt
```

El generador tomará esta solicitud y generará un TSTInfo. Que incluye el número de serie de la solicitud y la fecha del sello obtenida del Reloj

Unset

```
TSTInfo ::= SEQUENCE {
    version                INTEGER { v1(1) },
    policy                 TSAPolicyId,
    messageImprint         MessageImprint,
    -- MUST have the same value as the similar field in
    -- TimeStampReq
    serialNumber            INTEGER,
    -- Time-Stamping users MUST be ready to accommodate integers
    -- up to 160 bits.
    genTime                GeneralizedTime,
    accuracy               Accuracy OPTIONAL,
    ordering               BOOLEAN   DEFAULT FALSE,
    nonce                  INTEGER   OPTIONAL,
    -- MUST be present if the similar field was present
    -- in TimeStampReq. In that case it MUST have the same value.
    tsa                    [0] GeneralName OPTIONAL,
    extensions             [1] IMPLICIT Extensions OPTIONAL }

Accuracy ::= SEQUENCE {
    seconds                INTEGER   OPTIONAL,
    millis                 [0] INTEGER (1..999) OPTIONAL,
    micros                 [1] INTEGER (1..999) OPTIONAL }
```

<https://www.ietf.org/rfc/rfc3161.txt>

Firma

La TSA tomará el TSTInfo y lo firmará para convertirlo en un TimeStampResp, que ya es el sello de tiempo completado y firmado.

Esta firma se realizará utilizando un certificado guardado en el HSM y podrá ser:

1. Firma con digestión 2048 para sellos para firmas
2. Firma con digestión 4096 para sellos de conservación

El tipo de firma que se realice dependerá de la configuración de la TSA

Estructura de respuesta al Usuario:

```
Unset
HTTP 200
Content-Type: application/timestamp-reply
Content-Length: [longitud del cuerpo de la respuesta]

[TimeStampResp]
```

Estructura del TimeStampResp:

```
Unset
TimeStampResp ::= SEQUENCE {
    status                PKIStatusInfo,
    timeStampToken        TimeStampToken    OPTIONAL }

PKIStatusInfo ::= SEQUENCE {
    status                PKIStatus,
    statusString          PKIFreeText      OPTIONAL,
    failInfo              PKIFailureInfo  OPTIONAL }

PKIStatus ::= INTEGER {
    granted                (0),
    -- when the PKIStatus contains the value zero a TimeStampToken, as
    requested, is present.
    grantedWithMods        (1),
    -- when the PKIStatus contains the value one a TimeStampToken,
    with modifications, is present.
```

```
rejection          (2),
waiting            (3),
revocationWarning  (4),
-- this message contains a warning that a revocation is
-- imminent
revocationNotification (5)
-- notification that a revocation has occurred }
```

<https://www.ietf.org/rfc/rfc3161.txt>

Autenticación del solicitante y control de acceso

API

Se generará un par de credenciales WebId y ApiKey con el que se firmará la solicitud y se agregará esa firma como encabezado

Cada solicitud deberá tener un encabezado de autenticación que nos permitirá verificar que es un usuario autorizado y que la solicitud no ha sido cambiada por un tercero. El encabezado estará formado de la siguiente manera:

Identificador: el encabezado tiene que iniciar con "Authorization: HMAC-SHA256".

WebId: el WebId que fue asignado como parte de las credenciales

Separador: entre el WebId y el HMAC tiene que haber un símbolo ":".

HMAC: un SHA-256 HMAC en base64 generado con a partir del hash de la solicitud

```
Unset
POST HTTP
Content-Type: application/timestamp-query
Authorization: HMAC-SHA256 eyJhbGciOiJIUzI1NiIs...
```

Uso por Usuario:

El usuario pasará por un proceso de verificación con el proveedor de Identidad. Esto generará un token OAuth que tendrá que ser enviado en el header Authorization:

```
Unset
POST HTTP
Content-Type: application/timestamp-query
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Protección de la privacidad

La TSA debe proteger la privacidad del solicitante y garantizar que la información sellada no sea accesible para terceros no autorizados.

El RFC 3161 de la IETF (<https://www.ietf.org/rfc/rfc3161.txt>) define el funcionamiento de las estampillas de tiempo. Según este estándar, las solicitudes de sellos de tiempo sólo recibirán los HASH de la información que sellarán, por lo que:

1. La TSA nunca recibe la información que se está sellando, sólo su HASH
2. La información se encuentra segura aunque se llegaran a exponer los sellos generados de la TSA. Ya que aún teniendo el HASH, es imposible volver a la información original a partir del HASH.

Reloj

La TSA debe obtener el día y hora de una fuente confiable, para esto se usará el servicio de “Amazon Time Sync Service” sincronizado con Chrony

Construido sobre la probada infraestructura de red de Amazon, el servicio utiliza una flota de relojes de referencia satelitales y atómicos redundantes en las regiones de AWS para entregar lecturas de tiempo actuales del estándar global Tiempo Universal Coordinado (UTC). El servicio está diseñado para ser altamente disponible con una infraestructura de tiempo continuamente monitoreada y proporciona una fuente de tiempo de referencia con baja varianza. Los segundos intercalares son conocidos por causar errores en las aplicaciones, y pueden ser una preocupación para los desarrolladores y administradores de sistemas. El Servicio de Sincronización de Tiempo de Amazon suaviza automáticamente (difumina) los segundos intercalares que se añaden periódicamente al UTC, de manera que los clientes no tienen que preocuparse por los errores en las aplicaciones debido a su adición.

<https://aws.amazon.com/about-aws/whats-new/2017/11/introducing-the-amazon-time-sync-service/>

Registro de auditoría

El registro de auditoría guardará la siguiente información:

1. Timestamp de la solicitud: La fecha y hora exactas en que se recibió la solicitud de sello de tiempo.
2. Usuario: Identificador del usuario que realizó la solicitud
3. Dirección IP: Dirección desde donde se realizó la solicitud
4. Hash del documento: El hash del documento o datos para los que se solicitó el sello de tiempo.
5. Sello de tiempo emitido: El sello de tiempo que se generó y emitió en respuesta a la solicitud.
6. Firma del sello de tiempo: La firma generada por el HSM.
7. Timestamp de emisión: La fecha y hora exactas en que se emitió el sello de tiempo.

Esta información se podrá encontrar en las tablas TimeStampEvent y TimeStampResponse:

TimeStampEvent:

replyCreatedAt	timeStampDate	requestCreatedAt	requestHash	webId	..	hashOid
2023-08-29 23:59:39...	2023-08-29 23:59...	2023-08-29 23:59:31.7...	Ez7piSk/knNjASgMbxTInVI...	jc38kkX5bet98u	7	2.16.840.1.
2023-08-29 23:59:31...	2023-08-29 23:59...	2023-08-29 23:59:28.7...	Ez7piSk/knNjASgMbxTInVI...	jc38kkX5bet98u	6	2.16.840.1.
2023-08-27 02:59:07...	2023-08-27 02:59...	2023-08-27 02:59:06.8...	Ez7piSk/knNjASgMbxTInVI...	jc38kkX5bet98u	5	2.16.840.1.
2023-08-27 02:59:03...	2023-08-27 02:59...	2023-08-27 02:59:03.0...	Ez7piSk/knNjASgMbxTInVI...	jc38kkX5bet98u	4	2.16.840.1.

TimeStampResponse:

genTime	digest	hashAlgorithm	response	eventId
3 2023-08-26 19:...	Ez7piSk/knNjASgMbxTInVI...	2.16.840.1.101.3.4.2.1	0x3082090D30150201003010...	fb0ef4f-794c-4aca-bb06-a8e886d806e4
2 2023-08-26 19:...	Ez7piSk/knNjASgMbxTInVI...	2.16.840.1.101.3.4.2.1	0x3082090D30150201003010...	dde06dcf-9d84-493c-b8ed-36296a396308
1 2023-08-25 10:...	Ez7piSk/knNjASgMbxTInVI...	2.16.840.1.101.3.4.2.1	0x3082090D30150201003010...	deb8529f-4717-4937-9149-6bbcd32fb4ee
10 2023-08-30 13:...	Ez7piSk/knNjASgMbxTInVI...	2.16.840.1.101.3.4.2.1	0x3082090D30150201003010...	8c9e6523-1cf7-4f57-86ee-ea9d164efc46